AIM- To interface an HC_SR04 ultrasonic sensor with STM32F446RE and classify distance ranges using visual indication through LEDs.

APPARATUS- STM32 Nucleo-F446RE development board, HC-SR04 ultrasonic sensor, jumper cables, USB TypeA to MiniB cable, STM32 CUBE IDE, STM32 CubeMX.

THEORY

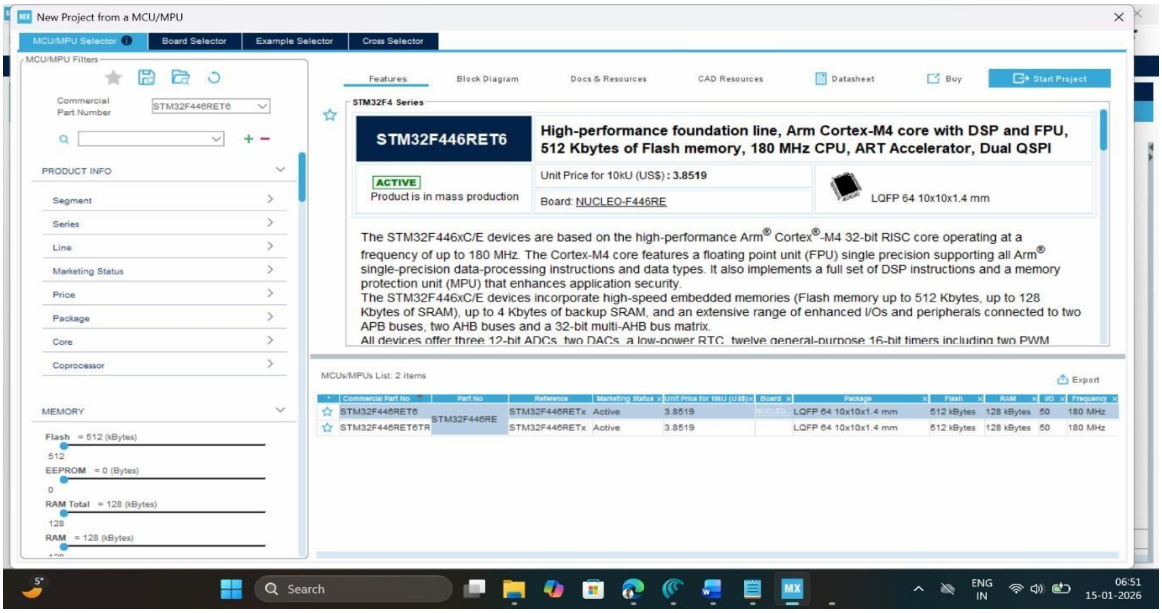
STM32F446RE provides multiple general-purpose I/O (GPIO) ports **(GPIOA–GPIOH)** that can be configured as input, output, alternate function, or analog via GPIO control registers or using the HAL library. When interfacing an

HC_SR04 ultrasonic distance sensor with the **STM32F446RE (ARM Cortex M4)**, specific microcontroller peripherals are used to translate the sensor's ultrasonic time-of-flight signals into meaningful distance data. Typically, one GPIO pin is configured as a digital output to generate the 10 µs trigger pulse, while another pin is configured as either a GPIO input or a timer input_capture channel to record the width of the echo pulse, which encodes the round _trip travel time of the ultrasonic wave. To calculate the distance, the STM32 uses a **General Purpose Timer (GPT)** configured in **Input Capture mode**. By capturing the timer counts at the rising and falling edges of the Echo pulse, the duration Δt is determined. The distance is then calculated using the speed of sound: $d = (v * \Delta t) / 2$

where v is approximately 343 m/s.

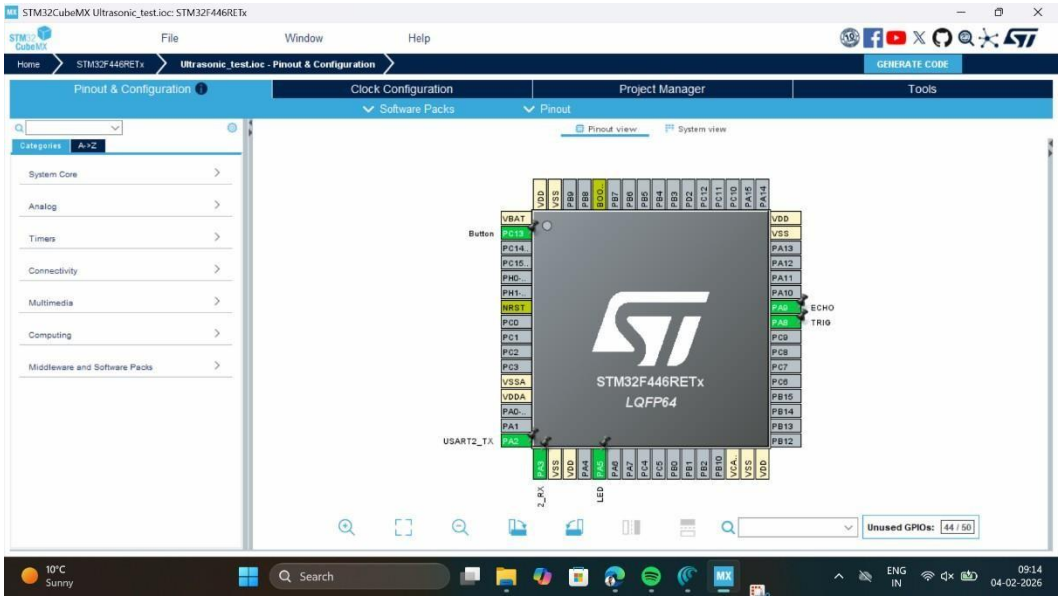
PROCEDURE

1. Open CubeMX and Click on ACCESS TO MCU SELECTOR.

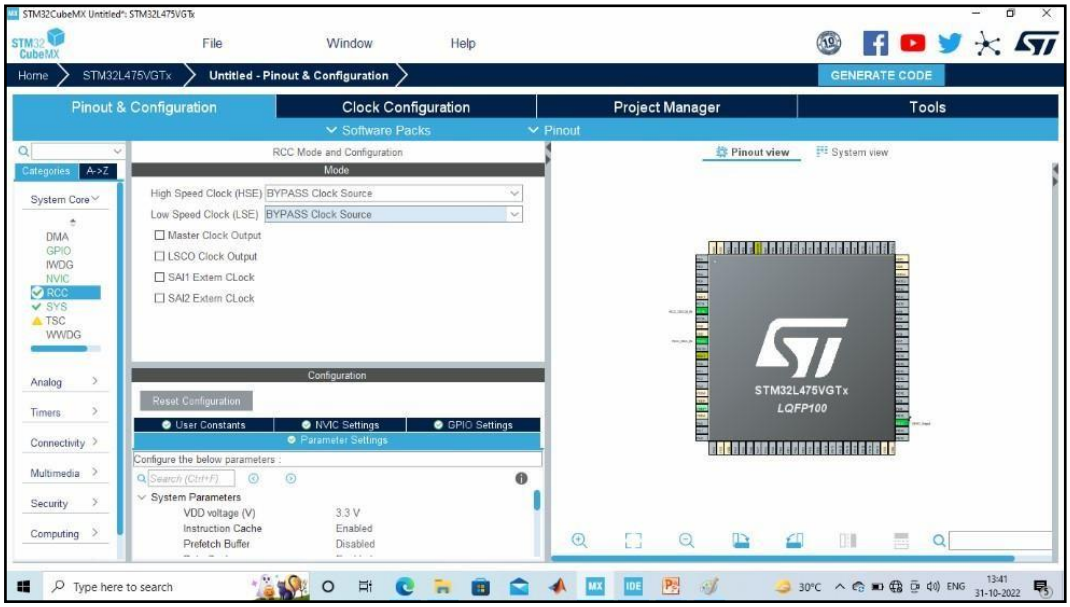


2. Write and select the microcontroller as you want to use and then click start the project. Select commercial part number STM32L446RE.
3. In system Core select GPIO and right click on pin PA5 to make it as GPIO output pin as shown below as LED (Inbuilt LED) and PC13 as Pushbutton (USER) relates to it in the board. Furthermore, initialize PA9 (ECHO) as

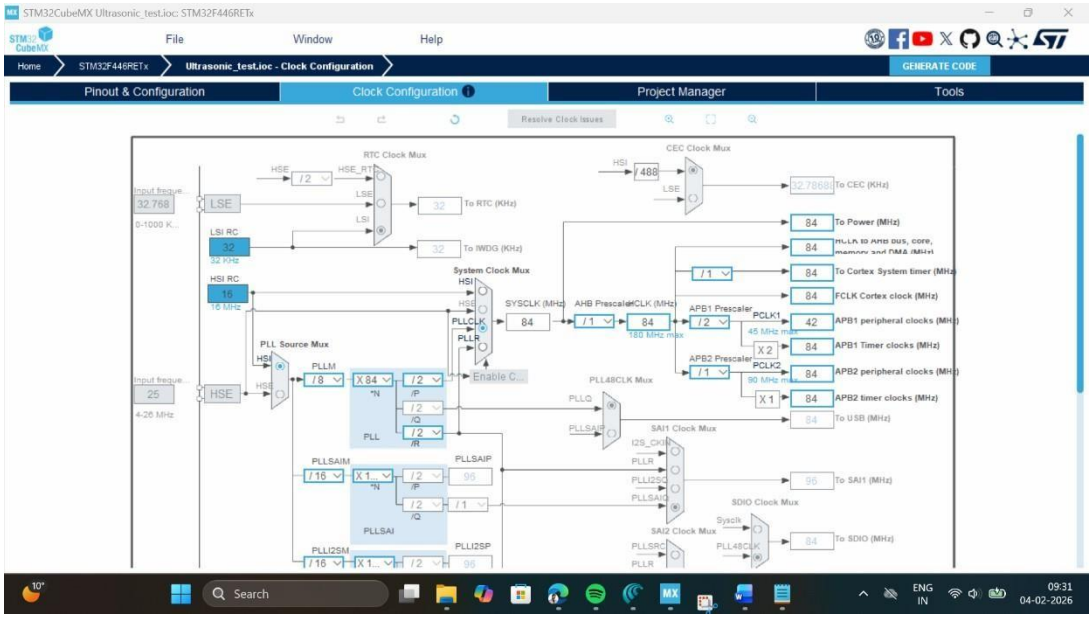
GPIO input, PA9 (TRIG) as GPIO output. Initialize USART 2 by selecting PA2 and PA3 pins as USART_Tx and USART_Rx pins for asynchronous mode.



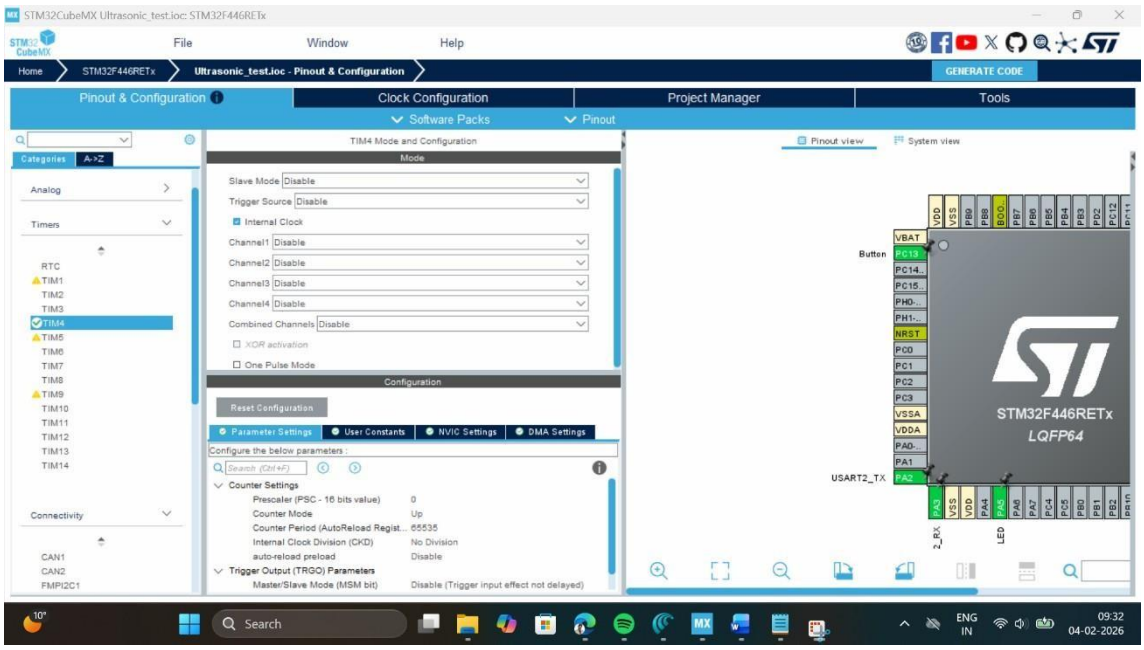
4. **Clock Selection**: Select the internal clock by clicking RCC and selecting BYPASS clock source



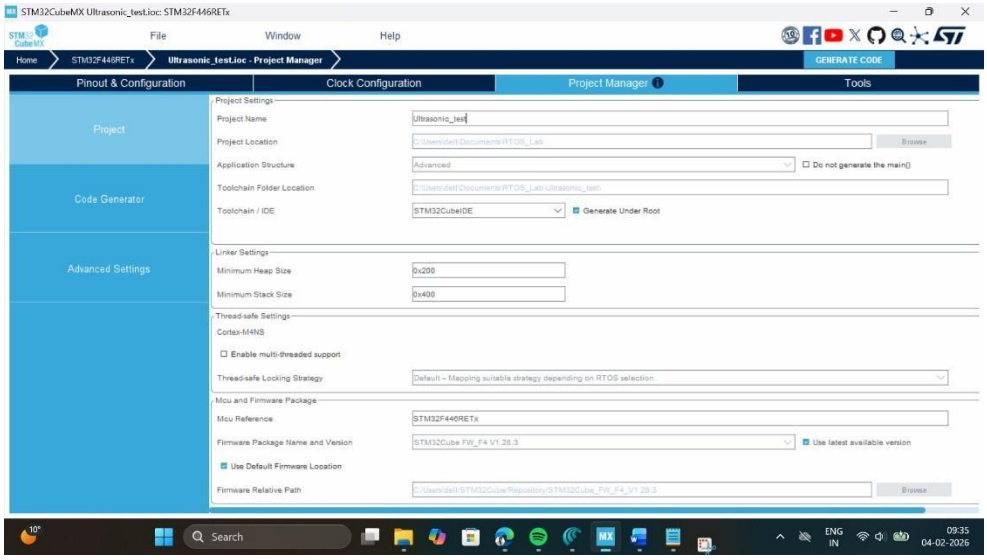
5. **Clock Configuration:** Configure clock as shown below to get maximum internal clock frequency of 84MHz.



6. Initialize general purpose time 4 TIM4 from timer section of project configuration and set it to internal clock. Set the Pre-scaler value to 84-1 so that timer can generate timing for 1us from the 84MHz of clock frequency signal.



7. Write Project name and select IDE.

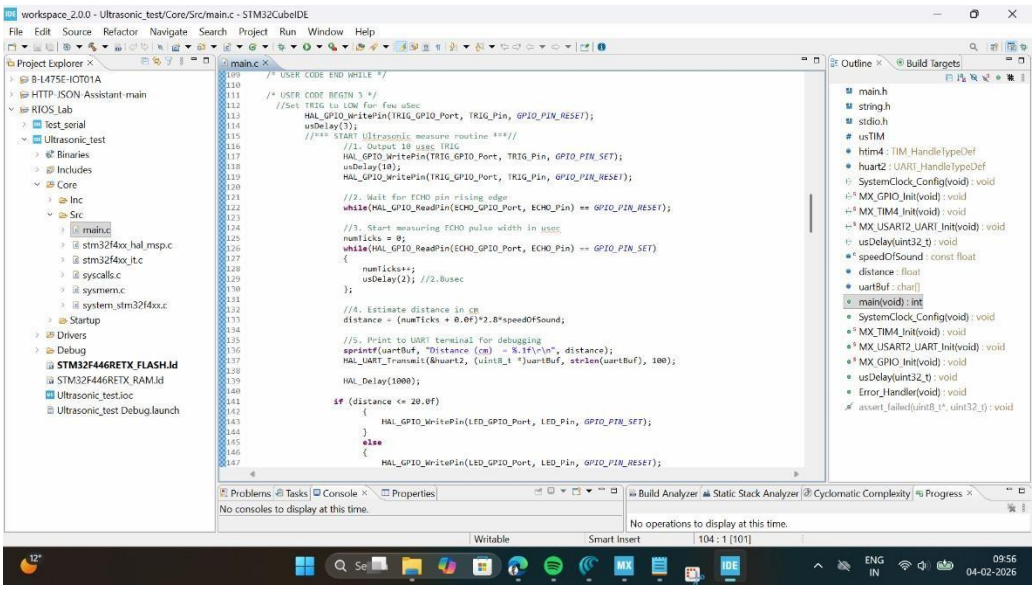


8. Click on generate code.
9. Click on open project.
10. Click ‘OK’ on the generated popup showing “successfully imported the project on the workspace” and the project will be successfully added to the cube IDE.
11. Open the main source code on Cube IDE by clicking on **main.c**
12. To generate bin file and hex file, right click on project and click properties.
13. Expand C/C++ Build, click on setting, click on MCU post build outputs and select convert to binary file as well as convert to hex file and click “Apply and Close ".

14. After configuration write following instructions in the **main.c** file and compile the program and ensure there will be zero error after compiling.

```
/* Private includes ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */
#include <string.h>
#include <stdio.h>
/* USER CODE END Includes */

/* Private typedef -----------------------------------------------------------*/
/* USER CODE BEGIN PTD */
#define usTIM TIM4
/* USER CODE END PTD */

/* USER CODE BEGIN PFP */ void
usDelay(uint32_t uSec);

/* USER CODE END PFP */

/* Private user code ---------------------------------------------------------*/
/* USER CODE BEGIN 0 */
//Speed of sound in cm/usec const float
speedOfSound = 0.0343/2;
float distance;
char uartBuf[100];
/* USER CODE END 0 */

int main(void)
{

 /* USER CODE BEGIN 1 */ uint32_t
       numTicks = 0;
 /* USER CODE END 1 */

 /* MCU Configuration--------------------------------------------------------*/

while (1)
 {
  /* USER CODE END WHILE */

  /* USER CODE BEGIN 3 */
        //Set TRIG to LOW for few uSec
        HAL_GPIO_WritePin(TRIG_GPIO_Port, TRIG_Pin, GPIO_PIN_RESET);
       usDelay(3);
        //*** START Ultrasonic measure routine ***//
        //1. Output 10 usec TRIG
```

```
  HAL_GPIO_WritePin(TRIG_GPIO_Port, TRIG_Pin, GPIO_PIN_SET);
usDelay(10);
  HAL_GPIO_WritePin(TRIG_GPIO_Port, TRIG_Pin, GPIO_PIN_RESET);

//2. Wait for ECHO pin rising edge
while(HAL_GPIO_ReadPin(ECHO_GPIO_Port, ECHO_Pin) == GPIO_PIN_RESET); //3.
Start measuring ECHO pulse width in usec     numTicks
= 0;
while(HAL_GPIO_ReadPin(ECHO_GPIO_Port, ECHO_Pin) == GPIO_PIN_SET)
 {
        numTicks++;
        usDelay(2); //2.8usec
        };

//4. Estimate distance in cm distance = (numTicks +
0.0f)*2.8*speedOfSound;

//5. Print to UART terminal for debugging sprintf(uartBuf,
"Distance (cm)  = %.1f\r\n", distance);
HAL_UART_Transmit(&huart2, (uint8_t *)uartBuf, strlen(uartBuf), 100);
HAL_Delay(1000);
if (distance <= 20.0f)
     {
       HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin, GPIO_PIN_SET);
     }
else
     {
    HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin, GPIO_PIN_RESET);
    }}

 /* USER CODE BEGIN 4 */ void
usDelay(uint32_t uSec)
{
    if(uSec < 2) uSec = 2;
    usTIM->ARR = uSec - 1;      /*sets the value in the auto-reload register*/
    usTIM->EGR = 1;                  /*Re-initialises the timer*/
    usTIM->SR &= ~1;            //Resets the flag
    usTIM->CR1 |= 1;            //Enables the counter
    while((usTIM->SR&0x0001) != 1); usTIM-
    >SR &= ~(0x0001);
}

/* USER CODE END 4 */
```



15. To run this program, connect the board with USB cable, click on build, click ‘ok’ and click ‘switch’ on popups.
16. Now you are in debugging mode, click ‘Resume’ to execute the program.

OBSERVATION:

S. No	Object Distance Actual (in cm)	Distance Measured UART (in cm)	on LED Status (On/OFF)	Echo Pulse width (µs)	Remarks
1	**5**	3.9	ON	99.12	Reading stable, small error.
2	**10**	8.3	ON	241.9	Measured value close to actual distance.
3	**15**	13.2	ON	384.8	Acceptable measurement error observed.
4	**20**	16.3	ON	475.2	Error starts in erasing by increasing distance
5	**25**	20.9	OFF	609.3	Crossed the acceptable range, LED OFF

RESULT

In this experiment, I successfully interfaced the HC-SR04 ultrasonic sensor with the STM32F446RE microcontroller and verified that the system was working properly. The sensor was able to measure the distance, and the calculated values were displayed correctly on the UART terminal.

From the observation table, I noticed that the measured distances were very close to the actual distances at shorter ranges such as 5 cm, 10 cm, and 15 cm, with only small errors. However, when the distance increased to around 20 cm and 25 cm, the error became slightly larger. This could be due to small timing inaccuracies or limitations of the sensor.

The LED also worked according to the condition set in the program. It turned ON when the measured distance was less than or equal to 20 cm and turned OFF when the distance was greater than 20 cm, showing that the logic in the code functioned correctly.

Overall, the system performed well. The distance was calculated using the time-of-flight formula,

$d = (v \times \Delta t) / 2$, where $v = 343$ m/s.